Why does sASM work — and only when the subdomain solve is inexact?

An interpretation through information propagation and overlap over-counting

---

Minimal 1D / 2D Laplace experiment (all numerics in PETSc): BASIC vs. sASM additive Schwarz, with *exact* (Cholesky) and *inexact* (ICC(0)) subdomain solves.

1D: 256 DOF, 8 subdomains. 2D: 64^2 grid, 4×4 subdomains.

The result that frames everything

CG iterations to $\|r\|/\|b\| < 10^{-8}$, same problem, only the subdomain solver changes

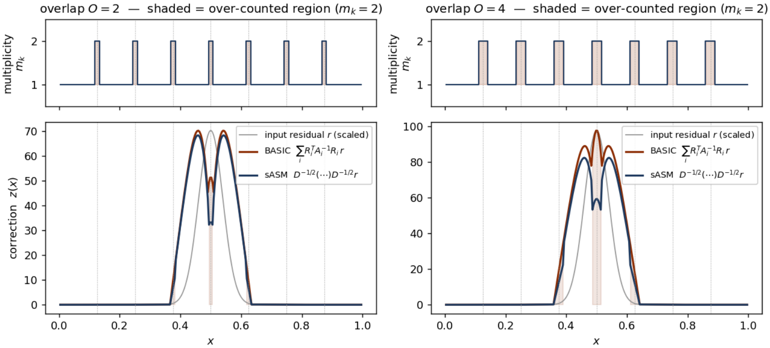
	1D Laplace		2D Laplace	
	exact	ICC(0)	exact	ICC(0)
BASIC	15	15	23	**102**
sASM	25	25	27	**67**

- **sASM beats BASIC only in one cell: 2D with an inexact solve** (67 vs 102). Everywhere else sASM $\approx$ or slightly worse than BASIC.
- 1D "ICC" $=$ exact: a tridiagonal matrix has no fill, so ICC(0) is the complete Cholesky. Hence 1D cannot show the anomaly at all.
- **Question:** what makes the *inexact* solve bad, and why does sASM fix it precisely there?

Mechanism, part 1: BASIC over-counts the overlap

one preconditioner apply $\sum_i R_i^\top A_i^{-1} R_i\, r$ to a localized residual

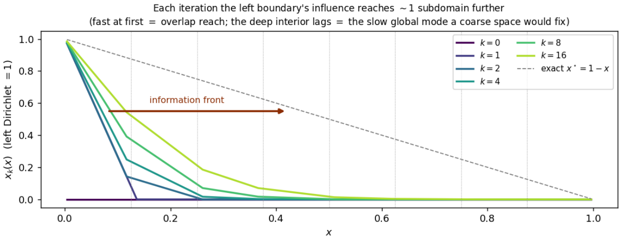
Fig. 1 — One preconditioner apply: BASIC over-counts the overlap; sASM normalizes it



- In the overlap a DOF belongs to m_k subdomains, so its correction is *added* m_k times.
- **BASIC** (red) bumps up at every overlap seam; **sASM** (blue) is normalized by $1/\sqrt{m_k}$.
- This over-count is the algebraic fact $\sum_i R_i^\top R_i = \mathbf{D} = \mathrm{diag}(m_k)$. It is harmless *by itself* — the next slides show when it bites.

Mechanism, part 2: information propagates one subdomain per iteration

g. 2 — **Information propagation in one-level Schwarz: a bounded distance per iteration** ⇒ O(#subdomains) **ite**



- Laplace is globally coupled, but each Schwarz step solves only *local* problems.
- So the boundary's influence advances ∼1 subdomain per iteration (fast at first = overlap reach).
- The deep interior lags — that slow global mode is what a coarse space (two-level) would remove.
- **One-level cost is O(#subdomains) iterations**, independent of BASIC vs sASM. Overlap helps by speeding this propagation.

Q1 — Why is the inexact (ICC) solve bad?

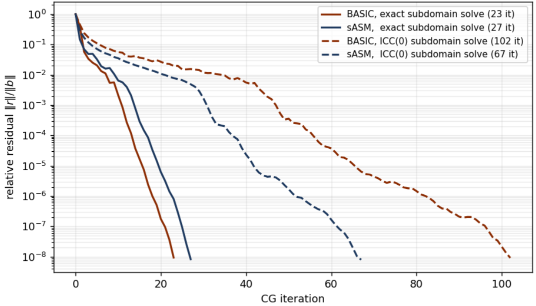
the abstract Schwarz bound: the two factors *multiply*

$$\kappa(\mathbf{M}^{-1}\mathbf{A}) \;\le\; C_0^2 \cdot \underbrace{\omega}_{\substack{\text{(A) inexactness}\\ \text{exact}=1,\ \text{ICC}>1}} \cdot \underbrace{N_\mathrm{c}}_{\substack{\text{(B) over-count}\\ (\text{colouring } /\ m_k)}}$$

- **Exact** ($\omega=1$): each local correction is the *exact* local solution; over-counting only lifts $\lambda_{\max} \le N_\mathrm{c}$ — bounded, CG tolerates it. *No anomaly* (table: 23 vs 27).
- **ICC** ($\omega>1$): each local correction carries an *error*, and BASIC **adds** m_k **of these errors** in the overlap. They do not cancel — they accumulate:
  - $\lambda_{\max} \le \omega\, N_\mathrm{c}$: the inexactness is *multiplied* by the over-count;
  - wider overlap makes the block bigger, so fixed-fill ICC is a worse approximation ($\omega\uparrow$) *and* $N_\mathrm{c}\uparrow$ — so κ *rises* with overlap (the anomaly).
- The accumulated, amplified error **piles up at the subdomain seams** and the outer CG must clear it.

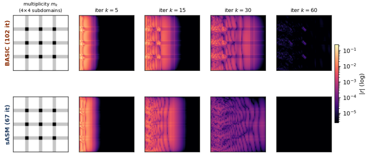
Q1 — the evidence

Fig. 3 — 2D Laplace: sASM beats BASIC ONLY with an inexact solve
(exact: 23 vs 27; ICC(0): 67 vs 102) — over-counting bites only when coupled with inexac



Residual history. Exact: BASIC≈sASM (both fast). ICC: BASIC **102** blows up.

Fig. 4 — ICC(0) subdomain solve: residual $|r|$ at MATCHED CG iterations. BASIC's residual lingers at the overlap seams; sASM clears it ⇒ 67 vs 102 iters



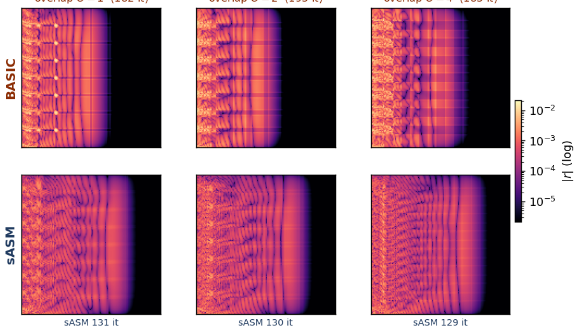
ICC residual field at matched CG iterations: at $k{=}60$ BASIC is still bright (error not cleared), the seams lit up by the over-counted ICC error.

"Inexact is bad" = the local error is **counted** m_k **times in the overlap** (A×B), accumulating into seam residual that costs iterations — and worsens as overlap grows.

Q1, deeper — the seam error grows with overlap

2D Laplace, 128^2, 8×8 subdomains, ICC(0); residual $|r|$ at fixed iteration $k=40$

Fig. 5 — Overlap sweep (128^2, 8×8, ICC(0), residual at fixed $k = 40$)**: BASIC's residual sits in the over-counted seams; the over-counted region (and the residual share trapped there,** 0.45→0.51→0.64**) widens with** O**; sASM clears it at every** O

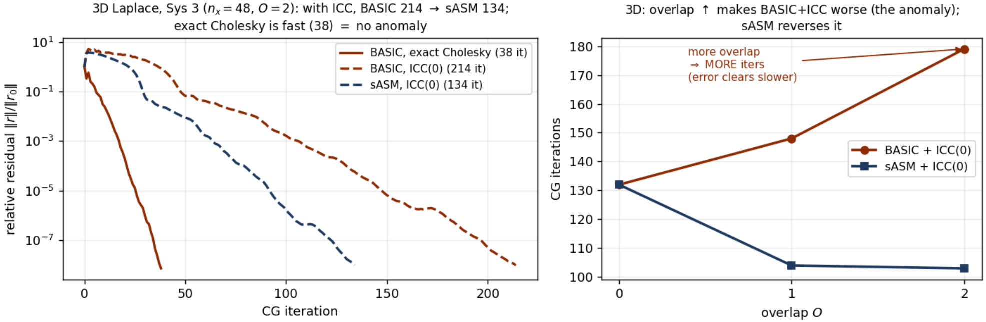


- The over-counted seam region *widens* with overlap, and the residual share trapped there grows $0.45 \to 0.51 \to 0.64$.
- sASM is better at *every* overlap (iters BASIC 182/195/185 vs sASM 131/130/129).
- Honest note: in 2D the iteration count vs O is non-monotone (ICC is only mildly inexact in 2D). The clean monotone "overlap↑⇒iter↑" is a 3D effect $\to$ next slide.

The same effect on the real 3D problem (Sys 3)

mixed Dirichlet/Neumann Laplace, $n_x\!=\!48$, 4 ranks — the actual torso-type solve

Fig. 6 — The same effect on the real 3D Sys 3 (mixed Dirichlet/Neumann Laplace)



- **Left:** ICC — BASIC 214 slow, sASM 134 recovers; exact Cholesky 38 (no anomaly, the reference).
- **Right:** the clean monotone anomaly — BASIC+ICC $132 \to 148 \to 179$ (more overlap, more iters = error clears slower); sASM $132 \to 104 \to 103$ reverses it.

Q2 — Why does sASM overcome it?

$\mathbf{M}_{\text{sASM}}^{-1} = \mathbf{D}^{-1/2}\big(\sum_i R_i^\top A_i^{-1} R_i\big)\mathbf{D}^{-1/2}$, **partition of unity** $\sum_i d_i^2 = 1$

$$\text{BASIC: } \lambda_{\max} \le \omega\, N_{\text{c}} \qquad \xrightarrow{\mathbf{D}^{-1/2}(\cdot)\mathbf{D}^{-1/2}} \qquad \text{sASM: } \lambda_{\max} \le \omega$$

- The $\mathbf{D}^{-1/2}$ sandwich normalizes the overlap so each DOF's correction is counted **once**: the over-count factor N_{c} is **removed** (Fig. 1, blue is flat).
- **This breaks the multiplication.** The inexactness ω is no longer multiplied by N_{c}: the ICC error in the overlap is counted once, *not accumulated* $\Rightarrow$ no seam pile-up.
- Result (ICC): the seam residual clears (Fig. 4, sASM nearly dark at $k{=}60$); CG recovers **102** $\to$ **67**.
- **sASM does not fix the inexactness** (ω is still > 1) — it stops the over-count from *amplifying* it. So $\kappa \sim C_0^2\,\omega$, and overlap helps again.

What does "exact / inexact subdomain solve" actually solve?

$A_i = R_i A R_i^\top$ — the global Laplace restricted to subdomain i (with overlap)

- A_i implicitly imposes **homogeneous Dirichlet** on the *artificial cut* between subdomains (the cross-interface coupling is dropped), and keeps the physical BC on faces that touch the real boundary.
- **Exact solve** A_i^{-1} (Cholesky) $=$ *exactly* solving a local Dirichlet–Laplace subproblem. $\omega = 1$.
- **Inexact solve** ICC(0) $=$ exactly solving a *perturbed* operator $\tilde{A}_i = A_i - E_i$, where E_i is the fill that ICC(0) *drops*. Equivalently, the same Dirichlet subproblem solved only approximately. This E_i **is** the source of $\omega > 1$, and its error sits in the overlap/seam.

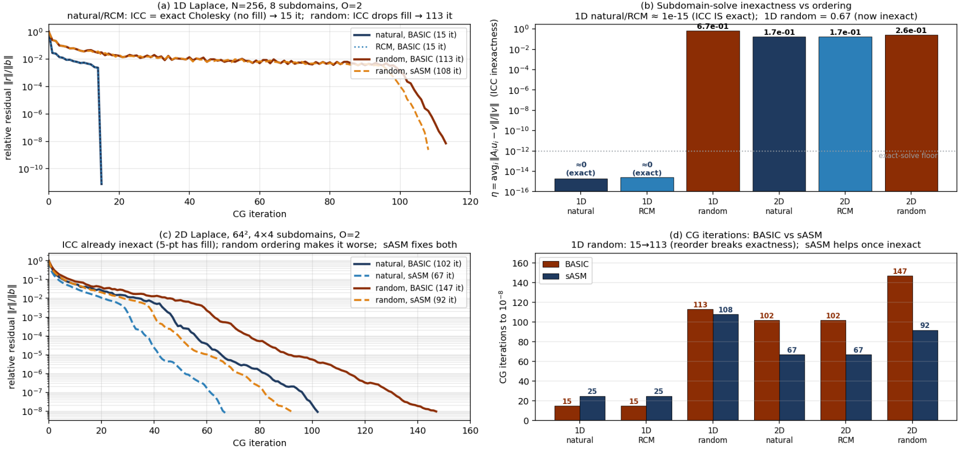
So "inexact" is not mysterious

ICC(0) keeps only A_i's nonzero pattern and throws away all fill-in. Whether that loses anything depends entirely on the **ordering** of the unknowns — next slide.

Reordering the unknowns turns the ICC inexact

only the ICC *ordering* changes (natural / RCM / random); subdomains, overlap, weights fixed

Fig. 7 — Reordering the subdomain unknowns turns the ICC factorization inexact (Q1: 1D natural ICC = exact; random ICC ≠ exact)



Reading Fig. 7 — reordering breaks (or keeps) exactness

$\eta = \mathrm{avg}_i \|A_i u_i - v\|/\|v\|$, **the inexactness proxy (one ICC solve of a random v; $\eta \approx 0$ = exact)**

- **(b) the headline.** 1D *natural* & *RCM*: $\eta \approx 10^{-15}$ — ICC *is* exact Cholesky, because a tridiagonal has *no fill*, so dropping fill drops nothing. 1D *random*: $\eta = 0.67$ — the permutation creates fill that ICC(0) throws away, so ICC is now inexact.
- **(a)/(d) 1D iterations.** natural/RCM converge in 15; random blows up to 113. RCM keeps the 1D problem banded, so it stays exact — *not every* permutation breaks exactness, only a band-widening (random) one.
- **(c)/(d) 2D.** The 5-point ICC already has fill (η=0.17); random makes it worse (η=0.26): $102 \to 147$ it. **sASM repairs both** ($102 \to 67$, $147 \to 92$), more so the more inexact the solve.

Conclusion

The one-sentence picture

BASIC counts the overlap twice (Fig. 1). With *exact* solves that is harmless — it only lifts $\lambda_{\max}$, which CG tolerates (Fig. 3, solid). With an *inexact* ICC solve the local error is counted m_k times, so "inexactness $\times$ over-count" accumulates at the seams (Fig. 4) and convergence degrades (Fig. 3, dashed, 102). sASM's $\mathbf{D}^{-1/2}$ partition of unity removes the over-count, breaking that product, so the inexact corrections are no longer amplified and CG recovers (102 $\rightarrow$ 67).

- **Q1:** inexactness is bad because BASIC *multiplies* it by the overlap over-count N_{c}; and both grow with overlap, so iterations rise with overlap.
- **Q2:** sASM removes N_{c} (partition of unity), so the inexactness is no longer amplified — it helps *exactly* when the solve is inexact, and is neutral when it is exact.
- Information always propagates $\sim$1 subdomain/iteration (Fig. 2): a one-level cost, untouched by either method.